

Plano de Curso: Docker e Cloud Computing para Desenvolvimento de Software

Docente: João Cardoso

23 de julho de 2026

Informações Gerais

- **Curso:** Análise e Desenvolvimento de Sistemas (ADS)
 - **Disciplina:** Arquitetura de Software e Computação em Nuvem
 - **Carga Horária Total:** 20 horas
 - **Duração:** 10 semanas (2 horas/aula + 1h prática individual)
 - **Estrutura por Aula:**
 - 30 min: Exposição teórica
 - 30 min: Prática guiada com instrutor
 - 60 min: Prática individual (fora da aula, com suporte online)
 - **Público-alvo:** Estudantes de ADS e desenvolvedores em início de carreira
 - **Pré-requisitos:** Conhecimento básico de Python, linha de comando e Git
- Sumário dos Assuntos e Plano do Curso:** <https://github.com/Zuluke/softwarearch-cloudcomputing>

Objetivo Geral

Capacitar o aluno a containerizar aplicações, orquestrar ambientes multi-container e realizar deploy na nuvem AWS, adotando boas práticas de mercado para desenvolvimento de software.

Ementa do Curso

1. Fundamentos de containers e virtualização
2. Docker: criação de imagens e containers
3. Docker Compose: ambientes multi-container
4. Registries e versionamento de imagens
5. Introdução à Computação em Nuvem (AWS)
6. Deploy de aplicações na AWS (ECS Fargate)
7. Armazenamento e persistência (S3, RDS)
8. Escalabilidade e requisitos não funcionais
9. CI/CD e automação de deploys
10. Projeto Final: Pipeline completo

Cronograma de Aulas

Aula 1 — Introdução a Containers e Virtualização

Parte 1: Exposição Teórica (30 min)

- O problema da consistência em equipes de desenvolvimento
- “*Funciona na minha máquina*” e seus custos
- VM vs Container: arquitetura e diferenças
- Casos de uso no mercado

Parte 2: Prática Guiada (30 min)

- Instalação do Docker Desktop
- Primeiros comandos: `docker run hello-world`
- Execução de containers pré-existent
- Gerenciamento de containers (`docker ps`, `docker stop`)

Prática Individual (1h)

1. Executar `docker run -it ubuntu bash` e explorar o container
2. Executar `docker run -d -p 80:80 nginx`
3. Acessar `http://localhost` no navegador
4. Explorar comandos: `docker ps`, `docker logs`, `docker exec`
5. **Desafio:** Rodar um container MySQL e conectar-se a ele

Aula 2 — Dockerfile e Criação de Imagens

Parte 1: Exposição Teórica (30 min)

- Estrutura do Dockerfile (FROM, WORKDIR, COPY, RUN, CMD)
- Gerenciamento de dependências com `requirements.txt`
- Boas práticas: camadas e cache
- `.dockerignore` e otimização de imagens

Parte 2: Prática Guiada (30 min)

- Criar uma aplicação Flask simples
- Criar Dockerfile para a aplicação
- Build da imagem: `docker build -t app .`
- Executar o container e testar

Prática Individual (1h)

1. Criar uma aplicação Python que calcula Fibonacci
2. Criar Dockerfile com Python 3.10-slim
3. Construir imagem e executar container
4. Modificar o código e rebuildar a imagem
5. **Desafio:** Criar multi-stage build para reduzir o tamanho da imagem

Aula 3 — Docker Compose e Ambientes Multi-Container

Parte 1: Exposição Teórica (30 min)

- Docker Compose: definição de serviços
- Arquivo `docker-compose.yml`
- Redes entre containers
- Volumes para persistência de dados

Parte 2: Prática Guiada (30 min)

- Configurar Flask + PostgreSQL com Docker Compose
- Subir ambiente: `docker-compose up -d`
- Conectar aplicação ao banco de dados

Prática Individual (1h)

1. Criar aplicação Flask com rota `/health`
2. Adicionar banco de dados PostgreSQL no compose
3. Adicionar Redis para cache
4. Configurar volumes para persistência
5. **Desafio:** Adicionar pgAdmin para gerenciar o banco

Aula 4 — Registries e Versionamento de Imagens

Parte 1: Exposição Teórica (30 min)

- Docker Hub: registro público de imagens
- Versionamento de imagens (tags)
- Docker Registry privado (ECR, Harbor, GitLab Registry)
- Boas práticas de versionamento

Parte 2: Prática Guiada (30 min)

- Criar conta no Docker Hub
- Taggear imagem: `docker tag app:latest usuario/app:v1`
- Push da imagem: `docker push usuario/app:v1`
- Pull e execução de imagem pública

Prática Individual (1h)

1. Criar conta no Docker Hub
2. Enviar imagem do projeto anterior para o Docker Hub
3. Criar tags para diferentes versões (v1, v2, latest)
4. Compartilhar imagem com colegas
5. **Desafio:** Configurar GitHub Actions para build e push automático

Aula 5 — Introdução à Computação em Nuvem (AWS)

Parte 1: Exposição Teórica (30 min)

- Modelos de serviço: IaaS, PaaS, SaaS
- Principais serviços AWS
- EC2, S3, VPC, IAM
- AWS Free Tier e boas práticas de custo

Parte 2: Prática Guiada (30 min)

- Criar conta AWS (ou usar playground)
- Configurar AWS CLI
- Criar bucket S3 e fazer upload
- Listar buckets com AWS CLI

Prática Individual (1h)

1. Configurar AWS CLI na máquina
2. Criar bucket S3 com nome único
3. Fazer upload/download de arquivos
4. Configurar políticas de acesso público
5. **Desafio:** Criar um script Python para interagir com S3 (boto3)

Aula 6 — AWS ECR e ECS Fargate

Parte 1: Exposição Teórica (30 min)

- ECR: Elastic Container Registry
- ECS: Elastic Container Service
- Fargate vs EC2 Launch Type
- Task Definitions e Services

Parte 2: Prática Guiada (30 min)

- Criar repositório ECR
- Build e push da imagem para ECR
- Configurar ECS Fargate

- Deploy da aplicação

Prática Individual (1h)

1. Criar repositório ECR
2. Enviar imagem do projeto Flask para ECR
3. Criar Task Definition no ECS
4. Deploy da aplicação no Fargate
5. Testar o endpoint público
6. **Desafio:** Configurar Application Load Balancer

Aula 7 — Armazenamento e Persistência (S3, RDS)

Parte 1: Exposição Teórica (30 min)

- Armazenamento de arquivos estáticos (S3)
- Bancos de dados gerenciados (RDS)
- Configuração de segurança e IAM
- Backup e recuperação

Parte 2: Prática Guiada (30 min)

- Configurar bucket S3 para arquivos estáticos
- Criar instância RDS PostgreSQL
- Conectar aplicação ao RDS

Prática Individual (1h)

1. Criar bucket S3 para upload de imagens
2. Configurar RDS PostgreSQL
3. Conectar aplicação Flask ao RDS
4. Configurar variáveis de ambiente para conexão
5. **Desafio:** Configurar S3 para hosting de site estático

Aula 8 — Escalabilidade e Requisitos Não Funcionais

Parte 1: Exposição Teórica (30 min)

- Escalabilidade Vertical vs Horizontal
- Auto Scaling e Load Balancing
- Requisitos não funcionais: disponibilidade, performance, segurança
- Otimização de custos (Spot instances, right-sizing)

Parte 2: Prática Guiada (30 min)

- Configurar Auto Scaling no ECS
- Monitoramento com CloudWatch
- Configurar Application Load Balancer
- Testar escalabilidade com carga

Prática Individual (1h)

1. Configurar CloudWatch Alarms
2. Configurar Auto Scaling baseado em CPU/Memória
3. Configurar Load Balancer
4. Gerar carga artificial e observar escalabilidade
5. **Desafio:** Otimizar custos usando Spot instances

Aula 9 — CI/CD e Automação de Deploys

Parte 1: Exposição Teórica (30 min)

- GitHub Actions e integração contínua
- Estratégias de deploy: Rolling, Blue-Green, Canary
- Infraestrutura como Código (IaC)
- Terraform e AWS CloudFormation

Parte 2: Prática Guiada (30 min)

- Configurar GitHub Actions para build da imagem
- Push automático para ECR
- Deploy automático no ECS
- Monitoramento do pipeline

Prática Individual (1h)

1. Criar repositório GitHub
2. Configurar GitHub Actions com workflow completo
3. Testar pipeline com push de código
4. Configurar deploy automático para ECS
5. **Desafio:** Implementar Blue-Green Deployment

Aula 10 — Projeto Final e Apresentações

Parte 1: Exposição Teórica (30 min)

- Revisão dos principais conceitos
- Arquiteturas de referência
- Boas práticas de mercado
- Dicas para o projeto final

Parte 2: Prática Guiada (30 min)

- Orientação para o projeto final
- Esclarecimento de dúvidas
- Revisão de entregáveis

Prática Individual (1h)

1. Finalização do projeto
2. Documentação
3. Gravação de demonstração
4. Submissão do trabalho final

Matriz de Avaliação

Critério	Descrição	Peso
Participação nas aulas	Engajamento e presença	10%
Prática individual (10 aulas)	Entrega das atividades semanais	30%
Desafios semanais	Resolução dos desafios propostos	20%
Projeto Final	Pipeline completo com documentação	30%
Apresentação do projeto	Demonstração e defesa técnica	10%

Projeto Final

Escopo

Desenvolver e deployar uma aplicação completa com os seguintes componentes:

1. **Aplicação:** API REST em Flask com rotas para CRUD de produtos
2. **Banco de Dados:** PostgreSQL em container ou RDS
3. **Cache:** Redis para otimização
4. **Containerização:** Docker e Docker Compose
5. **Deploy:** AWS ECS Fargate com Load Balancer
6. **CI/CD:** GitHub Actions para build e deploy automático
7. **Monitoramento:** CloudWatch para logs e métricas

Entregáveis

- Código fonte (GitHub)
- Dockerfile e docker-compose.yml
- Documentação (README.md)
- Prints/demonstração da aplicação rodando
- Pipeline CI/CD configurado
- Apresentação de 5 minutos (vídeo ou ao vivo)

Critérios de Avaliação

Critério	Peso
Funcionamento da aplicação	25%
Containerização correta	20%
Deploy na AWS	20%
CI/CD configurado	15%
Documentação	10%
Apresentação	10%

Bibliografia

- **DOCKER, INC.** Docker Documentation. Disponível em: <https://docs.docker.com/>
- **AMAZON WEB SERVICES.** AWS Documentation. Disponível em: <https://docs.aws.amazon.com/>
- **Poulton, Nigel.** Docker Deep Dive. Independently Published, 2022.
- **Kim, Gene; Humble, Jez; Debois, Patrick; Willis, John.** The DevOps Handbook. IT Revolution Press, 2016.
- **Freeman, Emily.** DevOps for Dummies. Wiley, 2019.
- **Hightower, Kelsey; Burns, Brendan; Beda, Joe.** Kubernetes: Up and Running. O'Reilly, 2022.

Material de Apoio

Cheatsheet Docker

```
# Imagens
docker build -t nome:tag .
docker images
docker rmi nome:tag
docker push usuario/repo:tag
docker pull usuario/repo:tag

# Containers
docker run -it --rm nome:tag
```

```
docker run -d -p 8080:80 nome:tag
docker ps
docker ps -a
docker stop container_id
docker start container_id
docker rm container_id
docker logs container_id
docker logs -f container_id
```

```
# Volumes
docker volume create nome
docker volume ls
docker run -v volume:/app ...
```

```
# Docker Compose
docker-compose up -d
docker-compose down
docker-compose logs -f
docker-compose ps
```

```
# Limpeza
docker system prune -a
docker system prune -a --volumes
```

Cheatsheet AWS CLI

```
# Configuração
aws configure
aws sts get-caller-identity
```

```
# ECR
aws ecr create-repository --repository-name app
aws ecr get-login-password | docker login --username AWS --password-stdin
```

```
# S3
aws s3 ls
aws s3 cp arquivo.txt s3://meu-bucket/
aws s3 sync ./pasta s3://meu-bucket/pasta
```

```
# ECS
aws ecs list-clusters
aws ecs describe-services --cluster meu-cluster --services app
aws ecs update-service --cluster meu-cluster --service app --force-new-deployment
```

Template Dockerfile

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

Template docker-compose.yml

```
version: '3.8'

services:
  web:
    build: .
    ports:
      - "5000:5000"
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/app
      - REDIS_URL=redis://redis:6379
    depends_on:
      - db
      - redis

  db:
    image: postgres:15
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=pass
      - POSTGRES_DB=app
    volumes:
      - postgres_data:/var/lib/postgresql/data

  redis:
    image: redis:alpine

volumes:
  postgres_data:
```

Glossário

Termo	Definição
Container	Ambiente isolado que executa uma aplicação com suas dependências
Docker Engine	Plataforma que gerencia a execução de containers
Dockerfile	Arquivo de instruções para construir uma imagem
Imagem	Pacote executável contendo código, bibliotecas e configurações
Registry	Repositório de imagens Docker (ex: Docker Hub, ECR)
ECS	Serviço AWS para orquestração de containers
ECR	Serviço AWS para armazenamento de imagens Docker
S3	Serviço AWS para armazenamento de objetos
RDS	Serviço AWS para bancos de dados relacionais gerenciados
Fargate	Serviço AWS para execução de containers sem gerenciar servidores
Volume	Mecanismo de persistência de dados em containers
Blue-Green	Estratégia de deploy com duas versões em produção alternadas
Canary	Estratégia de deploy com liberação gradual para um percentual de usuários
CI/CD	Integração e Entrega Contínua (Continuous Integration/Continuous Delivery)

Load Balancer	Serviço que distribui tráfego entre múltiplos servidores
Auto Scaling	Capacidade de ajustar automaticamente recursos conforme demanda

Observações para o Instrutor

- **Infraestrutura:** Garanta que todos os alunos tenham Docker instalado antes da primeira aula
- **AWS:** Considere usar AWS Educate ou créditos gratuitos para os alunos
- **Aulas práticas:** Prepare exemplos prontos para caso os alunos tenham dificuldade
- **Suporte:** Mantenha canal de comunicação aberto (Discord/Slack) para dúvidas entre aulas
- **Projeto final:** Incentive trabalhos em duplas para simular ambiente de equipe
- **Atualização:** Revise os exemplos antes de cada aula para garantir que funcionam